

4) End-to-end Runtime Flow

Last updated by | Mennell, Jacob | 19 May 2026 at 12:22 BST

End-To-End Runtime Flow

Purpose

This page explains how work moves through the solution from source documents and claim requests through ingestion, retrieval, AI execution, and user-facing delivery.

End-To-End Flow Summary

The platform operates as a staged pipeline:

1. claim-related content enters the system
2. source documents are transformed into structured, retrieval-ready chunks
3. chunks, metadata, and embeddings are persisted
4. workflow execution retrieves relevant evidence from stored chunks
5. extraction or checks logic runs with QA and bounded retry behavior
6. outputs are exposed through API endpoints and consumed by frontend pages or other clients

Detailed Runtime Sequence

1. Input and initiation

Work can be initiated through application usage or orchestration endpoints that trigger background processing for claims. The API provides orchestration routes for job execution, while the broader platform also includes document ingestion and claim data access paths.

2. Document ingestion and normalisation

The ingestion worker processes documents into a normalized representation. Based on the ingestion module, this includes:

- calling Azure Document Intelligence layout analysis
- reconstructing markdown-like text from the result
- identifying header hierarchy and section structure
- splitting content into retrieval-oriented chunks
- preserving special handling for markdown-style tables
- matching chunks back to page- and layout-level metadata where possible

If the DI-driven path fails or is intentionally bypassed, the code also supports a fallback ingestion path with reduced structural metadata.

3. Embedding and persistence

Each chunk is embedded and then persisted with:

- chunk text
- document and claim linkage
- document type

- vector embedding
- metadata such as header paths, section identity, page information, and layout information

This creates the retrieval-ready evidence base used by later AI workflows.

4. Retrieval assembly for AI tasks

When an extraction or check task runs, the main worker assembles evidence from persisted chunks. The retrieval layer supports multiple retrieval strategies, including lexical, semantic, and hybrid search patterns. Retrieved evidence is then organized into the execution state for the target task.

5. Execution routing in the main worker

The extraction architecture shows that runtime behavior is selected across multiple dimensions:

- retrieval mode determines the overall extraction pattern
- execution mode determines deterministic versus agentic behavior for a single pass
- QA acts as a downstream validation layer that can trigger retries

Depending on configuration, the system can follow:

- normal retrieval-backed execution
- progressive-stop execution
- progressive-fanin execution
- web-search-assisted execution

The checks architecture follows a similar pattern of explicit execution, QA evaluation, and bounded retry handling.

6. QA and bounded retries

After extraction or check execution, the system evaluates output quality. QA can request another pass when policy allows it, but retry behavior is intentionally bounded. For checks, the architecture also documents fail-soft behavior so recoverable failures do not always become hard workflow failures.

7. API exposure and frontend consumption

Once data and workflow outputs are available, the API exposes them through route groups for claims, documents, orchestration, models, feedback, and computation. The frontend consumes these capabilities through routed pages for claim views, extracted information, generic checks, model-specific pages, and administration.

Runtime Flow Diagram



sequenceDiagram

```
participant U as User or Client
participant FE as Frontend
participant API as API Service
participant MQ as RabbitMQ
participant IW as Ingestion Worker
participant DB as PostgreSQL + pgvector
participant MW as Main Worker

U->>FE: Open claim workflow
FE->>API: Request claim or orchestration action
API->>MQ: Queue background work when required
MQ->>IW: Trigger ingestion flow
IW->>DB: Persist chunks, metadata, embeddings
MQ->>MW: Trigger extraction or check flow
MW->>DB: Retrieve evidence
MW->>MW: Execute task, QA, bounded retries
MW->>DB: Persist outputs and telemetry
API->>DB: Read claim or workflow results
FE->>API: Fetch data for pages
API-->>FE: Return JSON responses
FE-->>U: Render results
```

Design Notes

- Ingestion is deliberately separated from workflow execution so document preparation can evolve independently from reasoning logic.
- Retrieval is treated as a first-class runtime concern rather than an incidental database lookup.
- QA is positioned downstream of execution so validation policy remains consistent even when execution mode changes.
- The API remains the stable surface for delivery and integration, while workers handle longer-running or asynchronous concerns.